Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Yordy Pineda Jiménez y Jhon Lojan
Asignatura	Teoría de la programación
Ciclo	Primero A
Unidad	Primer
Resultado de aprendizaje de la unidad	Identifica los conceptos fundamentales de la teoría de la programación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Práctica Nro.	00**5**
Título de la Práctica	Del diseño del algoritmo con estructuras secuenciales a la construcción del programa.
Nombre del Docente	José Oswaldo Guamán
Fecha	Jueves 29 de octubre del 2026 al viernes 30 de octubre del 2026
Horario	Jueves, 12h30 – 13h30 viernes, 09h30 - 11h30
Lugar	Aula física asignada al paralelo
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

- Desarrollar la capacidad de transformar un problema en una solución
- computacional.
- Aplicar estructuras secuenciales en el diseño del algoritmo en un lenguaje
- de programación.
- Validar la lógica del algoritmo mediante pruebas de escritorio

3. Materiales, Reactivos, Equipos y Herramientas

- Herramienta de Pseudocódigo: procesador de texto
- Computador personal con sistema operativo Windows
- Material de apoyo en el Entorno Virtual de Aprendizaje (EVA)

- Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF
- Conexión a internet estable para acceder a recursos digitales y software en línea.
- IDE de programación: Visual Studio Code u otro entorno compatible.
- Lenguaje de programación: Según los contenidos de la unidad.

4. Procedimiento / Metodología Ejecutada

Metodología de aprendizaje: aprendizaje basado en problemas.

Desarrollo:

Inicio

Análisis del problema

Un estudiante necesita saber qué calificación debe obtener en el tercer certamen (C3) para aprobar la asignatura con una nota final de 60/100 puntos.

2. Diseño de la solución:

```
Algoritmo NotaCertamen3
    // Definir variables y constantes
    C1 <- 0.0;
    C2 <- 0.0;
    NotaLaboratorio <- 0.0;
    NotaCertamen <- 0.0;
    C3 <- 0.0;
    NOTAAPROBATORIA <- 60.0;
    // Ingresar datos
    Leer C1;
    Leer C2;
    Leer NL;
    // Proceso
    NotaCertamen <- (NOTAAPROBATORIA - (NotaLaboratorio * 0.3)) / 0.7;
    C3 <- (3.0 * NotaCertamen) - C1 - C2;
    // Presentar información procesada
    Escribir "Resumen de notas y resultados"
    Escribir "Nota Certamen 1: %.2f\n", +C1;
    Escribir "Nota Certamen 2: %.2f\n", +C2;
    Escribir "Nota Laboratorio: %.2f\n", +NotaLaboratorio;
    Escribir "Promedio de certamenes necesario (NC): %.2f\n", +NotaCertamen;
    Escribir "Calificacion minima requerida en C3: %.2f\n", +C3;
    si (C3 < 0)
        Escribir "Cuenta con las notas suficiente para aprobar";
    finsi
FinAlgoritmo
```

Figura 1 Pseudocodigo

3. Codificación: implementar la solución en el lenguaje de programación.

```c
#include <stdio.h>
#define NOTAAPROBATORIA 60.0 // nota mínima final de aprobación
int main()
{
    float C1, C2, NotaLaboratorio, NotaCertamen, C3;
    // Ingresar datos
    printf("Ingrese la nota del certamen 1 (C1):\n");
    scanf("%f", &C1);
    printf("Ingrese la nota del certamen 2 (C2):\n");
    scanf("%f", &C2);
    printf("Ingrese la nota de laboratorio (NL):\n");
    scanf("%f", &NotaLaboratorio);
    // Proceso
    // Calcular nc
    NotaCertamen = (NOTAAPROBATORIA - (NotaLaboratorio * 0.3)) / 0.7;
    // encontrar c3 despejando la formula
    C3 = (3.0 * NotaCertamen) - C1 - C2;
    // Presentar
    printf("Resumen de Notas y Resultado \n");
    printf("Nota Certamen 1: %.2f\n", C1);
    printf("Nota Certamen 2: %.2f\n", C2);
    printf("Nota Laboratorio: %.2f\n", NotaLaboratorio);
    printf("Promedio de certamenes necesario (NC): %.2f\n", NotaCertamen);
    printf("Calificacion minima requerida en C3: %.2f\n", C3);
    if (C3 < 0)
    {
        printf("Cuenta con las notas suficiente para aprobar");
    }
}
```

Figura 2 Programado en C

4. Pruebas de funcionamiento: ejecutar el programa y comprobar que los resultados sean correctos (casos de prueba).

```
Ingrese la nota del certamen 1 (C1):
70
Ingrese la nota del certamen 2 (C2):
80
Ingrese la nota de laboratorio (NL):
90
Resumen de Notas y Resultado
Nota Certamen 1: 70.00
Nota Certamen 2: 80.00
Nota Laboratorio: 90.00
Promedio de certamenes necesario (NC): 47.14
Calificacion minima requerida en C3: -8.57
Cuenta con las notas suficiente para aprobar
```

Figura 3 Presentacion resultados

Nota Certamen 1 (C1)	70.00	60.00
Nota Certamen 2 (C2)	80.00	60.00
Nota Laboratorio (NL)	90.00	60.00

Promedio de certámenes necesario (NC)	47.14	60.00
Calificación mínima requerida en C3	-8.57	60.00
Mensaje adicional	Cuenta con las notas suficiente para aprobar	No se muestra

Tabla 1 Prueba escritorio

5. Resultados

Se logro completar con éxito dando solución al problema planteado, usando las formulas que el ejercicio plantea, modificándolas para que nos den el valor requerido, además de identificar si al ser un valor negativo cuenta con la nota suficiente para aprobar.

6. Preguntas de Control

• ¿Qué es un programa con estructuras secuenciales?

Un programa con **estructuras secuenciales** es aquel cuyas líneas de código e instrucciones se ejecutan estrictamente en **orden lineal, paso a paso y una tras otra**, desde el inicio hasta el fin del programa [1].

En este tipo de estructura, la salida o resultado de una instrucción sirve a menudo de base para la siguiente, sin que existan saltos, desviaciones ni repeticiones en el flujo de ejecución [1].

• ¿Cómo se traslada un algoritmo en pseudocódigo o diagrama de flujo a un lenguaje de programación?

1. Validación previa del diseño algorítmico

Antes de escribir cualquier línea en el lenguaje destino, el algoritmo debe pasar por las fases estrictas de **análisis del problema**, **desarrollo del pseudocódigo** y **verificación mediante la prueba de escritorio** [1]. La prueba de escritorio consiste en simular paso a paso la ejecución con valores concretos para comprobar que las operaciones arrojen los resultados esperados [1]. Tras garantizar que la lógica está libre de errores, se procede formalmente a codificarlo en el lenguaje elegido (como C, Python o Java) [1].

2. Mapeo o traducción directa de componentes

El proceso de codificación consiste en sustituir cada instrucción general del pseudocódigo o símbolo del diagrama por las palabras reservadas y reglas sintácticas del lenguaje destino [1]:

● ¿Por qué es importante validar el programa mediante casos de prueba?

Para comprobar que funcionamiento del programa sea el correcto, y asegurar que funcione para lo que fue creado, además, de verificar errores que no se han visualizado a simple vista.

7. Conclusiones

- Se desarrolló la capacidad de analizar problemas y transformarlos en soluciones computacionales mediante la identificación de datos de entrada, procesos y resultados esperados.
- Se comprendió la importancia de aplicar estructuras secuenciales en el diseño de algoritmos, estableciendo un orden correcto para las instrucciones.
- Se comprobó la lógica y el funcionamiento de los algoritmos mediante pruebas de escritorio, lo que permitió verificar los resultados obtenidos, identificar posibles errores y comprobar que las instrucciones se ejecutaran correctamente de acuerdo con los datos ingresados.

8. Recomendaciones

Leer bien el problema que se pide solucionar, plantear primero una solución y así al momento de realizar el programa se tiene claro los pasos a seguir hasta donde se quiere llegar al programa.

9. Bibliografía / Referencias

Referencias

[1] M. M. A. Martínez., Lógica de programación con PSeInt. Enfoque práctico, 1.ª edición (diciembre de 2023) ed., entidad editorial: Colombia: Corporación Universitaria Remington (Fondo Editorial Remington), 2023.

[2] “Alternativa online a PSeInt Codoritmo,” Codoritmo. https://www.codoritmo.com/es

[3]M. Clow, “Visual Studio Code,” in Apress eBooks, 2018, pp. 57–68. doi: 10.1007/978-1-4842-3279-8_5.

10. Anexos

https://github.com/yordy2069/PrimerCicloU.git